Build Plan: A South-Indian–First Calorie & Exercise Tracker (MERN + Expo, v1)

TL;DR

- **Data:** Use USDA FoodData Central (free, CC0 public domain, REST) as your generic backbone, seed a local MongoDB `foods` collection from the India-specific **IFCT 2017** dataset (528 raw food items) and the **Anuvaad INDB** (1,014 cooked Indian recipes incl. dosa/idli/sambar/rasam) for South Indian coverage, and lean on your custom-recipe builder for family recipes. Skip paid APIs (Edamam/Nutritionix/FatSecret) for v1.
- **Architecture:** A pnpm-workspaces + Turborepo monorepo with `apps/api` (Express), `apps/web` (React 19/Vite/Tailwind), `apps/mobile` (Expo), and `packages/shared` (Zod schemas + inferred TypeScript types shared across all three) — the same shape as your CaterPro setup.
- **Build order for Claude Code:** Scaffold monorepo + shared types → auth/profiles → food DB seeding/search → custom recipe builder → daily food logging → exercise logging → dashboard/reports, driven by a spec-per-phase workflow with a lean root CLAUDE.md plus per-package CLAUDE.md files.

Key Findings

A) Nutrition data — the Indian/South-Indian coverage problem

The mainstream nutrition APIs are US-centric. USDA FoodData Central is free and public domain but has thin coverage of prepared Indian dishes. To make South Indian food a first-class citizen — a stated must-have — you should combine a generic source with India-specific datasets seeded into your own database rather than depend on a live third-party API:

- **IFCT 2017 (Indian Food Composition Tables)** — the authoritative Indian government nutrient tables authored by T. Longvah, R. Ananthan, K. Bhaskarachary and K. Venkaiah, © 2017 National Institute of Nutrition (NIN), ICMR, Hyderabad. It covers **528 raw food items** (the `nodef/ifct2017` package packages 542, "based on direct measurements across six regions"), across 151 food components, with names in 17+ Indian languages. Available as a downloadable dataset (Zenodo/Kaggle) and as the `ifct2017` npm package. Ideal for raw ingredients: rice, urad dal, coconut, tamarind, curry leaves, spices. GitHub Scribd
- **Indian Nutrient Databank (INDB)** by Anuvaad Solutions — per Vijayakumar et al. 2024 (*Current Developments in Nutrition* 8:103790), it comprises "a database of the nutrient composition data of individual food items (n = 1095) and a database of commonly consumed recipes (n = 1014)," with values per 100g and per serving. This is the single best source for ready-computed South Indian *dish* nutrition (dosa, idli, sambar, rasam, upma, poha, biryani,

chutneys). Downloadable as an Excel file; funded by the Bill & Melinda Gates Foundation and published open access under CC BY. PubMed ScienceDirect

- **USDA FoodData Central** — the APIs.io provider record describes "nutrient composition data for 600,000+ foods across Foundation Foods, SR Legacy, Survey Foods (FNDDS), Branded Foods, and Experimental Foods... All data is released under CC0 1.0 Universal (public domain)." The API Guide confirms the free-key limit: "FoodData Central currently limits the number of API requests to a default rate of 1,000 requests per hour per IP address." Best for generic/global and branded items. FoodData Central USDA FoodData Central
- **Open Food Facts** — a crowd-sourced database that (per world.openfoodfacts.org, 2025) has "added 4,000,000+ products from 150 countries"; useful for barcode/branded Indian packaged goods. Licensed under the **Open Database License (ODbL)** — attribution **and share-alike**. Open Food Facts Openfoodfacts
- **Paid APIs** — Edamam, Nutritionix, and FatSecret all offer Indian coverage of varying quality but come with cost, caching/storage restrictions, and attribution rules that make them overkill for a personal app (details below).

B) How existing calorie trackers model their data

The clearest open-source reference is **SparkyFitness** — a self-hosted, explicitly family-focused MyFitnessPal alternative (Node.js backend, PostgreSQL, React front-end, deployed via Docker Compose: Postgres + Node + NGINX). Its schema is a strong template and it solves your exact problem (multiple profiles, custom foods, multiple data providers incl. USDA/Open Food Facts/FatSecret/Nutritionix). Other useful references: **OpenNutriTracker** (custom meals + reusable recipe builder + categorised activity catalogue with direct-kcal entry) and **PANTS** (recipe-of-recipes nutritional analysis). XDA Developers + 2

C) Recommended architecture

A pnpm + Turborepo monorepo with shared Zod schemas is the current best practice for an Express + React + Expo stack. Per the Expo docs, "Expo has first-class support for monorepos managed with package managers supporting workspaces: Bun, npm, pnpm, and Yarn," and that support has been first-class since **SDK 53 (2025)**, with SDK 55 adding a `--workspace-root` flag for EAS Build. Expo Documentation React Native Relay

D) Claude Code workflow

Use a two-tier CLAUDE.md structure (lean root + per-package files), spec-driven development (plan mode → spec → numbered tasks → implement one at a time with review), and scope each session to a single package.

Details

A) Nutrition data sources — comparison & recommendation

Source	Coverage of Indian/S. Indian	Access	Cost	License	Verdict
IFCT 2017	Excellent for raw Indian ingredients (528 foods; 542 in npm pkg); multilingual	Downloadable dataset (Zenodo/Kaggle) + `ifct2017` npm package	Free	Package: MIT (older versions) / **AGPL-3.0 (current)**; underlying data © NIN/ICMR	**Seed locally**
INDB (Anuvaad)	Best for cooked S. Indian dishes (1,014 recipes)	Excel download	Free	"Open-access"; companion paper CC BY 4.0; no standalone dataset license file	**Seed locally**, attribute
USDA FDC	Weak on Indian dishes; strong generic/branded	REST API (1,000 req/hr) + bulk download	Free	CC0 (public domain)	**Generic fallback**
Open Food Facts	Decent for packaged Indian products	REST API + bulk	Free	ODbL (attribution + **share-alike**)	Optional (barcodes)
Edamam	Moderate; NLP ingredient parsing	REST/GraphQL	Free tier → paid	Attribution required; caching restricted	Skip for v1
Nutritionix	US/restaurant focus; weak generic Indian	REST	No public free tier (retired)	Commercial	Skip for v1
FatSecret	Good international (per-country); Indian market must be enabled	REST/OAuth	Free basic (5,000 calls/day, US dataset only); Premier by quote	Attribution on free/basic; **content must be deleted/refreshed within 24h** on non-indefinite tiers	Skip for v1

Recommendation: For a self-hosted personal/family app, do **not** build v1 around a live third-party API for Indian food — coverage and licensing both work against you. Instead:

1. **Seed a local MongoDB `foods` collection** from IFCT 2017 (raw ingredients) + INDB (cooked dishes). This gives you offline, fast, license-clean South Indian coverage from day one and makes South Indian dishes genuinely first-class.
2. **Add USDA FDC as an on-demand search fallback** (cache results into your `foods` collection) for generic/global/branded items not in the Indian datasets. The 1,000-requests/hour free key is ample for a handful of users.
3. **The custom recipe builder** (your requirement 2) covers everything else — family recipes computed from ingredient nutrition.

Licensing notes to act on:

- `ifct2017` **npm package:** the current version is **AGPL-3.0**; older versions are MIT. For a private, non-distributed family app either is fine, but AGPL's network-copyleft would require you to open-source your whole app if you ever publicly distribute/host it for others. Safer path: **pin an MIT version, or import the raw IFCT CSV/Zenodo data directly** and credit NIN/ICMR yourself. GitHub
- **INDB:** promoted as "open-access" and the companion paper (Vijayakumar et al. 2024) is CC BY 4.0, but the *dataset itself* has no standalone license file. Use with clear attribution (cite Vijayakumar et al. 2024 + Anuvaad + the underlying IFCT 2017/NIN-ICMR); email Anuvaad to confirm if you want certainty before any public release.
- **Both** ultimately trace their nutrient values to the **IFCT 2017 book, © National Institute of Nutrition / ICMR** — attribute NIN/ICMR regardless of source.

B) Data model — MongoDB schema

SparkyFitness (PostgreSQL) uses these core tables which map cleanly to MongoDB collections: `users`, `food_items`, `food_diary` (log entries with quantity, unit, meal_type, consumed_at), `exercises` (catalog), exercise logs, `family_access` (grantor/grantee/access_type), and goals. Recommended MongoDB collections: Codewithcj

`users` — profile, auth, per-user goals (daily calorie/macro targets), body metrics (weight for MET calc), and a `familyGroupId` so a handful of profiles share one instance.

`foods` (the searchable database) — one document per food/ingredient/dish:

```
{
  _id, name, aliases: [String],        // "dosa", "thosai", "dose"
  source: "IFCT2017" | "INDB" | "USDA" | "custom",
  sourceId, foodGroup, region,         // e.g. "South Indian"
  servingUnits: [{ label: "1 idli", grams: 30 }, { label: "100 g", grams: 100 }],
  nutrientsPer100g: { calories, protein, carbs, fat, fiber, ... },
  isVerified, createdBy, isPublic
}
```

Add a text index on `name` + `aliases` for search. Multilingual aliases from IFCT let users find foods by regional names.

`recipes` (custom/family recipes) — the fallback path:

```
{
  _id, name, createdBy, servings: Number,
  ingredients: [{ foodId, quantity, unit, grams }],
  totalNutrients: {...},               // computed on save/update
  nutrientsPerServing: {...},
  isPublic
}
```

Compute per-serving nutrition with the **Atwater method**: sum each ingredient's `nutrientsPer100g × (grams / 100)`, then divide totals by `servings`. Store the computed values so logs don't recompute. (Defer PANTS-style "recipe as an ingredient of another recipe" to a later version.) Healthcalculatoronline

`foodLogs` (daily diary):

```
{ _id, userId, date, mealType: "breakfast"|"lunch"|"dinner"|"snack",
  itemType: "food"|"recipe", itemId, quantity, unit, grams,
  nutrientsSnapshot: {...},            // denormalized at log time
  loggedAt }
```

Snapshot nutrients at log time so later edits to a food/recipe don't rewrite history.

`exercises` (catalog) + **`exerciseLogs`**:

```
exercises:    { _id, name, category, met: Number, source }
exerciseLogs: { _id, userId, date, exerciseId, durationMin,
                caloriesBurned, loggedAt }
```

For v1, compute calories burned as **MET × weight(kg) × duration(hr)**, or allow direct manual kcal entry (as OpenNutriTracker does). No workout generator — deferred per scope.

C) Tech architecture & monorepo

Monorepo layout (pnpm workspaces + Turborepo):

```
caltrack/
  CLAUDE.md              # root: structure, package manager, shared conventions
  turbo.json
  pnpm-workspace.yaml
  apps/
    api/       CLAUDE.md  # Express backend
    web/       CLAUDE.md  # React 19 + Vite + Tailwind
    mobile/    CLAUDE.md  # Expo / React Native
  packages/
    shared/    CLAUDE.md  # Zod schemas + inferred TS types + shared logic
```

Why pnpm + Turborepo: current best practice for RN monorepos; Metro/Expo have first-class monorepo support since SDK 53 (SDK 55 adds `--workspace-root` for EAS Build). pnpm gives hoisting control; Turborepo caches builds/tests. This matches the CaterPro-style structure you already run, so the learning curve is minimal. (React Native Relay)

Code/type sharing: Put **Zod schemas** in `packages/shared` and derive TypeScript types with `z.infer`. The Express API imports them for request/response validation and the React/Expo clients import the same schemas for forms (`@hookform/resolvers/zod`) — so "the shared Zod schemas live in a package that both the app and the API import from, so a schema change fails CI in both at once." Internal packages should be plain TypeScript (`"main": "./src/index.ts"`), **not** pre-built; Metro transforms them and Turborepo handles caching. (npm + 2)

Footguns to document in CLAUDE.md: exactly one copy of `react`/`react-native` (pin at workspace root; audit with `pnpm why react`, or you get "Invalid hook call" at runtime); a shared package must never import app code (a silent reverse dependency breaks the build graph — forbid with an eslint no-restricted-paths rule); set Metro `watchFolders`/`nodeModulesPaths` to the workspace root or you get "Unable to resolve module." (React Native Relay + 4)

Deployment (cheap/self-host): Docker Compose with MongoDB + Express (SparkyFitness ships exactly this pattern). Host API + web on a small VPS or low-tier PaaS; ship the mobile app via Expo/EAS. **MongoDB Atlas free tier (M0)** is sufficient for a family. Because it's not enterprise scale, you can skip Kubernetes, message queues, and horizontal scaling entirely.

D) Planning the build with Claude Code (2026)

CLAUDE.md structure — two tiers:

- **Root CLAUDE.md** (keep under ~200 lines): monorepo map, package manager (pnpm), workspace commands (`pnpm --filter`, `turbo run`), universal conventions (TS strict, named exports, conventional commits), and the shared-types rule. **Lead with the exact test/build/lint commands** — the highest-ROI section. Use HTML comments for human notes (stripped before hitting context).
- **Per-package CLAUDE.md** in each app and in `packages/shared`: stack-specific conventions (e.g. API: "route handlers in `src/routes`, business logic in `src/services`, validate all inputs with shared Zod schemas"). Claude Code reads up the tree and merges root + local automatically when you start a session inside the package. Claude Code Guides
- **Scope sessions to the package:** run Claude from the package directory (`cd apps/api && claude ...`) so file reads and commands are bounded to it and it uses the right test/build commands. Use git worktrees + parallel agents only for genuinely cross-package features.
- Don't duplicate what a linter/formatter enforces; prefer `file:line` references over pasted code; move task-specific detail into `.claude/rules/` or spec files loaded on demand.

Spec-driven workflow: In 2026, Claude Code's harness (plan mode, automatic compaction, persistent memory) handles most context management, so the developer's job shifts from context-wrangling to outcome specification. Use **plan mode** for each architectural phase, produce a short spec (what / why / acceptance criteria), turn it into numbered tasks, then implement one task at a time with a review between steps. GitHub Spec Kit or the Superpowers plugin formalize this loop but are optional for a solo build. Note the documented limitation: Claude Code can still drift from CLAUDE.md instructions, so keep specs short and verify outcomes yourself. Medium + 2

Suggested phased build order (one Claude Code "spec" per phase, in dependency order):

1. **Scaffold** — monorepo (pnpm + Turborepo), `packages/shared` with first Zod schemas (`User`, `Food`), root + per-package CLAUDE.md, Docker Compose with MongoDB, a `/health` endpoint. Verify all three apps boot and share types.
2. **Auth & profiles** — email/password (or OIDC) auth, JWT, a family group holding multiple profiles, per-user goals. The foundation everything else references.
3. **Food database seeding + search** — a seed script importing IFCT 2017 + INDB into `foods`; `/foods/search` with a text index on name + aliases; USDA FDC on-demand lookup that caches results into `foods`.
4. **Custom recipe builder** — ingredient picker (searches `foods`), quantity/unit entry, Atwater per-serving computation, save to `recipes`.
5. **Daily food logging** — diary UI with meal sections, log foods or recipes with quantity, nutrient snapshot at log time, daily totals vs goals.
6. **Exercise logging** — seed an exercise catalog with MET values, log duration → calories burned (MET formula or manual kcal).
7. **Dashboard/reports** — daily/weekly calorie & macro charts, goal progress, per-profile views.

In each phase, build/extend the shared Zod schemas **first** (`packages/shared` → `apps/api` → clients), since Claude Code updates consumers automatically when the shared contract changes.

Recommendations

1. **Start with local-seeded data, not a live Indian-food API.** Seed IFCT 2017 + INDB into MongoDB in Phase 3. This is the decisive choice that makes South Indian food first-class, works offline, and avoids licensing/caching headaches. Add USDA FDC only as a cache-on-demand fallback.
2. **Pin an MIT version of `ifct2017` or import the raw CSV/Zenodo data** rather than the current AGPL-3.0 package, to keep your options open if you ever share the app beyond family. Attribute NIN/ICMR; for INDB also cite Vijayakumar et al. 2024 + Anuvaad.
3. **Clone and study SparkyFitness first.** It's the closest existing reference (family, self-hosted, custom foods, multiple providers, Node-based) — its schema and provider-integration approach will save you design time.
4. **Adopt pnpm + Turborepo + `packages/shared` Zod schemas from day one** — same shape as CaterPro and the current RN-monorepo best practice.
5. **Drive the build as 7 specs, one per phase, in dependency order,** using plan mode and per-package CLAUDE.md files. Keep the root CLAUDE.md lean (<200 lines) and lead it with exact commands.

Thresholds that would change these recommendations:

- If you later want **natural-language logging** ("two idlis and sambar") or **restaurant/branded Indian coverage**, revisit FatSecret (per-country Indian dataset, enabled per market) or Nutritionix — but only once v1 is stable and only if the added cost/attribution is worth it.
- If you decide to **distribute the app publicly**, resolve the AGPL/IFCT data-license questions before shipping (pin MIT, confirm INDB terms with Anuvaad, honor NIN/ICMR attribution).
- If profiles grow **beyond a family (dozens+)**, move MongoDB off the free Atlas M0 tier and add proper API rate limiting and auth hardening.

Caveats

- **Data accuracy:** IFCT/INDB values are per-100g lab measurements; home portions and cooking (water loss/absorption) shift real calories by roughly 2–3× (e.g., 100g raw rice ≈ 360 kcal vs 100g cooked ≈ 130 kcal). Always store serving sizes in grams, let users adjust, and note "raw" vs "cooked." SparkyFitness's own docs warn that provider data isn't always accurate — cross-referencing is wise.
- **Licensing is not fully clean** for the Indian datasets (AGPL vs MIT version of the npm package; INDB lacks a standalone license file). Fine for private family use with attribution; get explicit confirmation before any public distribution.

- **Some figures come from secondary/marketing sources.** API pricing and coverage numbers are drawn partly from vendor and aggregator blogs (Suggestic, trybytes, YMove, Eat Fresh Tech) that may be dated or promotional; verify current pricing on each provider's own page before committing. The USDA, Expo, FatSecret-terms, and peer-reviewed INDB figures cited here are from primary sources.
- **Claude Code specifics shift between releases** — hook/skill config syntax and plan-mode behavior changed across 2025–2026 versions; check the current code.claude.com docs when you scaffold.
- **v1 scope** deliberately excludes the workout generator and other advanced features per your instruction; MET-based calorie burn is intentionally simple and approximate.